

蒙特卡洛光线追踪项目报告

一、开发环境与库依赖

本部分是当前项目中 Monte Carlo Path Tracer 的技术报告，内容主要在于路径追踪，而不是 Vulkan 实时光栅化。

- **操作系统:** Windows 10 / 11 64-bit
- **编程语言:** C++ (ISO C++17 Standard)
- **构建系统:** CMake
- **核心程序:** PathTracer.exe
- **并行加速:** OpenMP
- **CPU性能:** AMD 9700X
- **第三方库:**
 - **tinyobjloader:** 加载 OBJ/MTL
 - **stb_image:** 读取纹理
 - **stb_image_write:** 输出 PNG
 - **GLFW / Vulkan SDK:** 仍在仓库中，但主要服务于 VulkanApp，不是 PathTracer 主渲染链路的核心依赖

本项目实际是一个“双渲染器”仓库：

1. PathTracer.exe：离线路径追踪，负责全局光照、阴影、反射、折射、Monte Carlo 采样。
2. VulkanApp.exe：软光栅化 + HZB 的实时实验程序。

当前这份报告只讨论 PathTracer。

项目部署：

1. 编译

```
# MSVC
cmake --preset windows-msvc-release
cmake --build --preset windows-msvc-release --parallel

# MinGW
cmake --preset windows-mingw-release
cmake --build --preset windows-mingw-release --parallel
```

2. 运行 PathTracer

```
# Usage: PathTracer <scene_dir> [spp] [--set PT_KEY=VALUE] [--unset PT_KEY]
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\living-room 16

# 输出线性 HDR 结果
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\living-room 64 --set PT_WRITE_LINEAR=1
```

3. 批量测试

```
python tools\batch_render_scenes.py ".\build\windows-msvc-release\PathTracer.exe"
```

用户操作

PathTracer.exe 是离线渲染程序，没有实时交互窗口；用户主要通过命令行参数与环境变量控制渲染行为。

常用控制方式：

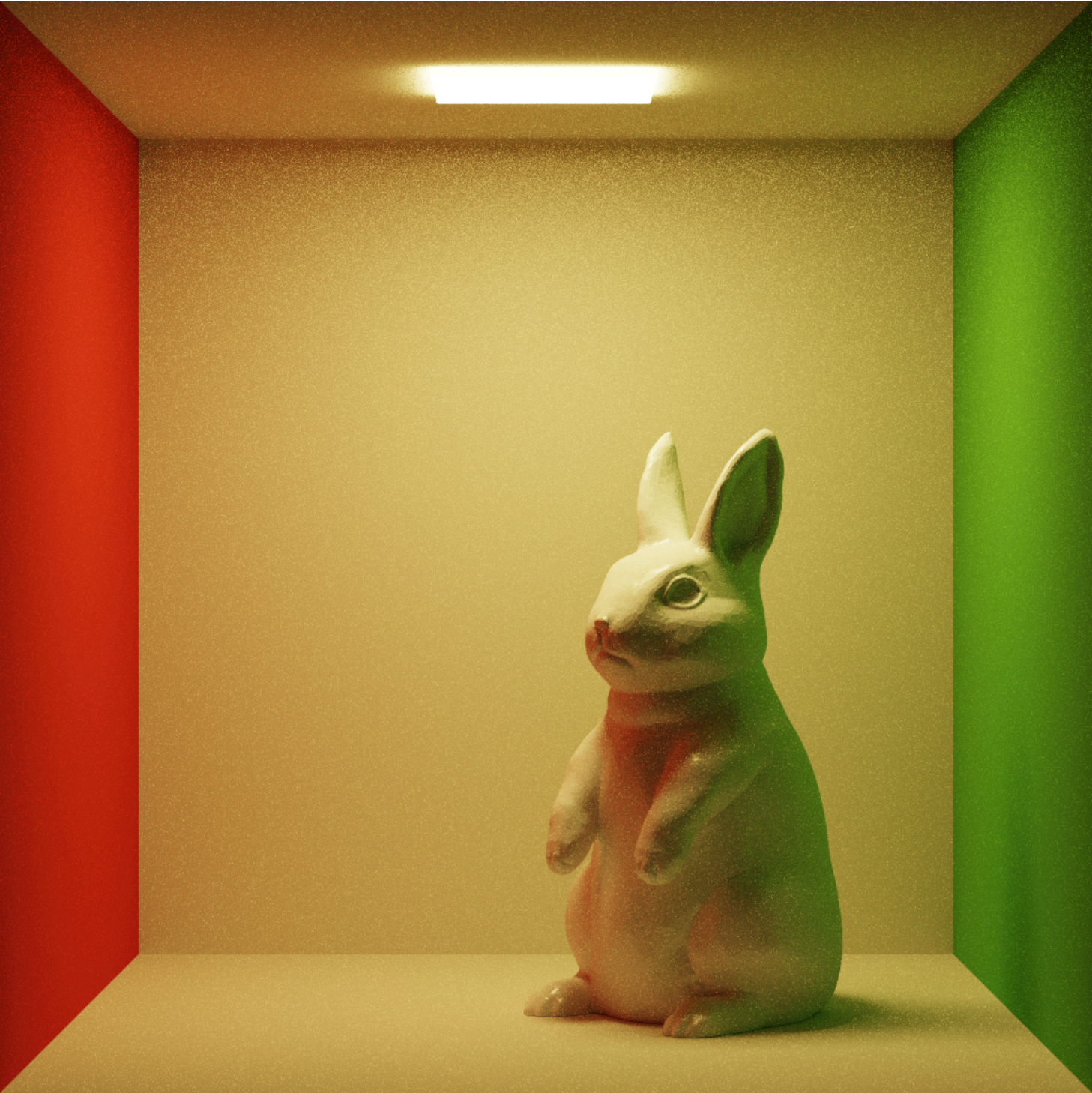
- **场景选择:** 传入场景目录，例如 example-scenes-cg25\cornell-box
- **采样率:** 第二个参数 spp 控制每像素采样数
- **调试开关:** 用 --set PT_XXX=... 控制 NEE、环境光、像素级 trace 等行为
- **输出格式:** 默认输出 PPM + PNG ，可通过 PT_WRITE_LINEAR=1 额外输出 PFM

例如：

```
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\sponza 16 --set PT_DEBUG_PIXEL=640,360
```

二、渲染测试数据

cornell-box_256

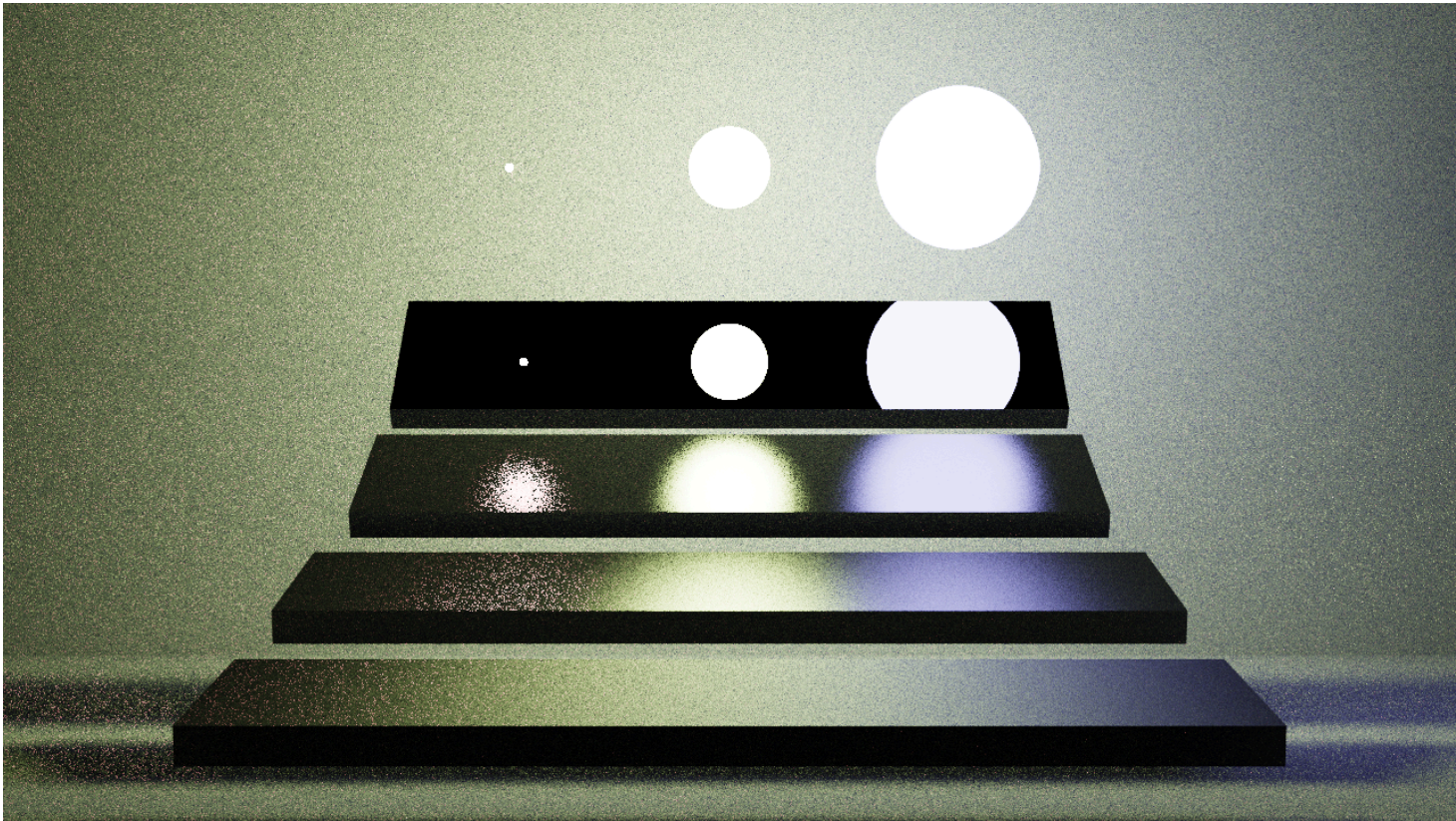


living-room_128



sponza_128





场景基础信息

场景	三角形数	材质数	BVH 节点数	发光三角形数	光源总面积	分辨率	备注
cornell-box	26,678	7	18,809	2	0.1786	1024x1024	OBJ normal warning
living-room	143,175	20	97,209	12	24	1280x720	检测到 enclosing emissive shell，自动缩放 0.01
sponza	262,267	25	176,087	2	100	1280x720	无 scene.xml ， fallback camera/environment，自动缩放 0.01，添加 procedural softbox
veach-mis	2,218	8	1,501	2,166	15.5433	1280x720	OBJ normal warning

不同采样率下的耗时

场景	1 spp	2 spp	8 spp	16 spp	32 spp	64 spp	128 spp	256 spp
cornell-box	16.36s	32.83s	129.60s	254.11s	510.19s	1011.40s	2017.69s	4038.11s
living-room	19.41s	36.54s	145.23s	286.90s	571.13s	1140.89s	2287.87s	进行中

场景	1 spp	2 spp	8 spp	16 spp	32 spp	64 spp	128 spp	256 spp
sponza	26.08s	50.70s	199.57s	402.19s	803.86s	1616.79s	3210.07s	进行中
veach-mis	7.95s	15.55s	61.12s	123.44s	245.89s	490.34s	976.74s	进行中
(稍微未跑完， 跑完我会更新)								

三、算法实现与模块设计

本项目实现的是一个以三角形网格场景为输入的 CPU 路径追踪器，核心流程是：

1. 加载场景几何、材质、纹理、相机与光源
2. 构建 BVH 加速结构
3. 对每个像素发射多条相机光线
4. 在路径追踪主循环中进行求交、直接光采样、BRDF 采样与路径延拓
5. 经曝光、色调映射和 sRGB 变换后输出图像

1. 代码模块划分

模块	主要文件	职责
Math	src/math/vec.h , src/math/ray.h	向量、矩阵、光线与常量
Scene	src/scene/mesh.h , src/scene/loader.cpp , src/scene/bvh.cpp	场景对象建模、OBJ/MTL/XML 加载、BVH 构建与遍历
Sampling	src/sampling/rng.h , src/sampling/brdf.cpp , src/sampling/light.cpp	随机采样、BRDF、相机发射、直接光采样
Integrator	src/integrator/pathtracer.cpp	路径追踪主循环
Output	src/output/tonemap.cpp	曝光、tonemap、PNG/PPM/PFM 输出
Main	src/pt_main.cpp	并行调度、进度统计、命令行参数处理

2. 核心数据结构

1. Vec3 / Ray
 - Vec3 在本项目里同时承担几何向量与颜色/辐射值表达。
 - Ray 非常轻量，仅包含 origin + direction 与 at(t)。
2. Material / Texture / Triangle / HitRecord
 - Material 包含 Kd/Ks/Ns/Tr/Ni/emission 以及纹理索引。
 - Texture 支持颜色采样与 alpha-mask 采样。
 - Triangle 是路径追踪最小几何单元。
 - HitRecord 保存命中距离、位置、着色法线、几何法线、UV、材质编号。
3. Scene / SceneFull
 - Scene 聚合材质、纹理、三角形与光源采样辅助数据。
 - SceneFull 继承 Scene ，并按值拥有 BVH bvh 。

- 当前真实运行链路以 `SceneFull::bvh` 为准。

4. BVH / AABB

- `AABB::hit(...)` 用 slab 法做光线与包围盒求交。
- `BVH::buildRec(...)` 根据最长轴 + 中点划分递归构建。
- `traverseClosest(...)` 与 `traverseShadow(...)` 分别服务于最近命中与遮挡判断。

3. 场景加载设计

场景加载入口位于 `src/scene/loader.cpp`，其工作顺序为：

1. 尝试解析 `scene.xml` 获取相机与区域光信息
2. 加载 `scene.obj`，若不存在则回退到 `<folder>.obj`
3. 构建材质、纹理和三角形数据
4. 对法线、透明贴图、特殊材质做兼容处理
5. 对数据集场景执行 fallback 相机与环境设置
6. 自动缩放场景尺度
7. 构建 BVH
8. 建立光源索引与 CDF

这种装配方式的优点是：所有渲染依赖在 `loadScene(...)` 结束后都已经就绪，后续积分器不再关心资源初始化细节。

4. 材质与 BRDF 模型

当前路径追踪器采用的是较为朴素但工程上稳定的材质模型：

- **Diffuse**: Lambert
- **Specular Glossy**: Phong lobe
- **Mirror**: 通过 `isMirror()` 判定，走解析反射分支
- **Glass**: 通过 `isGlass()` 判定，走折射/反射分支
- **Emissive**: 通过 `emission` 控制发光面

`src/sampling/brdf.cpp` 中的三个函数构成同一套概率模型：

1. `evalBRDF(...)`
2. `pdfBRDF(...)`
3. `sampleBRDF(...)`

三者一致是 Monte Carlo 正确性的基础，否则会造成偏差、噪点异常或亮度错误。

5. 路径追踪主循环

主循环位于 `src/integrator/pathtracer.cpp` 的 `pathTrace(...)`。

每一跳的主要步骤是：

1. 用 `sc.bvh.intersect(...)` 求最近命中
2. miss 时累积环境光
3. hit emissive 时累积面光贡献
4. 对非镜面/玻璃材质执行：
 - 直接光采样 `sampleDirectLight(...)`

- 环境直接采样 `sampleDirectEnvironment(...)`
- BRDF 采样生成下一跳

5. 使用 Russian Roulette 在较深路径处做概率终止

路径吞吐量更新形式为：

```
throughput = throughput * brdf * cosT / (pdf * pCont);
```

它对应的是标准路径积分估计器中的 $f * \cos / \text{pdf}$ 结构，并乘以 RR 存活概率修正。

6. 直接光采样与 MIS

`src/sampling/light.cpp` 中实现了两种直接光估计：

- `sampleDirectLight(...)`：对面积光源采样
- `sampleDirectEnvironment(...)`：对环境光采样

二者都结合 BRDF PDF 使用 MIS 的 power heuristic 融合。其作用是：

1. 降低纯路径追踪在小光源下的高方差问题
2. 减少“打不中光源只能靠碰运气”的低效率采样方式
3. 让面积光与环境光都能通过统一的估计框架参与贡献

7. 输出链路

`src/output/tonemap.cpp` 提供完整输出链：

1. `computeExposure(...)`：自动曝光，也可由 `PT_EXPOSURE` 强制指定
2. `tonemap(...)`：采用 ACES 风格曲线把 HDR 压缩到显示范围
3. `toSRGB(...)`：线性空间转 sRGB
4. `writePPM(...)` / `writePNG(...)` / `writePFM(...)`：输出文件

因此图像亮暗不完全由光照强弱决定，还会受曝光统计与色调映射影响。

四、实现细节与取舍

1. BVH 的必要性与并行策略

对于几十万三角形以上场景，若每次反弹都线性遍历全部三角形，成本无法接受。因此项目中必须使用 BVH。

当前 BVH 的特点：

- 构建策略简单稳定：最长轴 + 中点划分
- 遍历接口区分为最近命中与阴影遮挡两种路径
- 在 `src/scene/bvh.cpp` 中仅对浅层大子树启用 OpenMP task

这种并行策略不是最激进的，但优点是风险低、容易维护，并且避免了过度并行带来的调度开销。

2. 像素并行而不是路径内部并行

src/pt_main.cpp 采用：

- OpenMP collapse(2) 对二维像素循环并行
- 每像素独立 RNG
- 原子统计进度

这种设计的原因是：

1. 像素之间天然独立
2. 避免共享状态带来的锁与数据竞争
3. 比“路径内部细粒度并行”更容易验证正确性

3. 数据集兼容不是只支持课程格式

最初课程场景更偏向 scene.xml + scene.obj 固定结构，但当前 loader.cpp 已兼容：

- <folder>.obj 命名方式
- 缺少 scene.xml 的数据集目录
- 自动相机与环境回退
- Sponza 等场景的特殊比例与补丁光源处理

这使得 cornell-box、living-room、veach-mis、sponza 等目录都能通过统一入口运行。

4. 调试开关的工程意义

项目中大量使用环境变量控制行为，例如：

- PT_DISABLE_NEE
- PT_ONLY_NEE
- PT_ONLY_EMISSIVE_HIT
- PT_DEBUG_PIXEL
- PT_DEBUG_MAX_SAMPLES
- PT_DEBUG_MAX_BOUNCES
- PT_EXPOSURE
- PT_TARGET_GRAY
- PT_WRITE_LINEAR

这些参数的作用不是光线追踪的本职工作，而是把路径贡献拆开做受控实验，便于定位白斑、黑块、过曝、漏光、误判 miss 等问题。

五、Bug分析与Debug记录

1. BVH AABB 边界命中问题

项目中曾出现 interior 场景局部白斑问题，根因是 AABB slab 求交在边界接触时过于严格，导致部分原本应命中的光线被错误判定为 miss，随后直接落到环境光分支，形成不合理的亮斑。(核心是在src/scene/bvh.h 的 AABB::hit(...) 里，原来是 <=, 现

在改成 <

```
if(tmax < tmin) return false;)
```

这个问题的修复点在 `src/scene/bvh.h`，本质是避免把边界接触情况错误排除。

2. Cornell 右墙黑块与法线/偏移问题

在 Cornell 场景中，黑块问题不是单一参数可修复的，而是由以下因素共同造成：

- 脏法线或低质量法线输入(课程给的数据法线有问题)
- 法线与几何法线不一致导致半球采样异常
- 阴影/散射射线偏移方向不稳，触发自遮挡

工程处理方案是：

1. 对低面数平面材质做更强的法线稳定
2. 对高面数模型保留平滑法线，避免破坏外观
3. 修正偏移与 front/back 法线方向逻辑

3. Sponza 全黑问题

Sponza 场景的一个关键问题来自 `loader.cpp` 对 MTL 透射参数的解释错误。

Sponza 的很多普通材质虽然写有：

```
d 1.0000
Tr 0.0000
Tf 1.0000 1.0000 1.0000
```

但这里的 `Tf` 是透射颜色，不代表“材质是玻璃”。如果简单把 `Tf` 平均值直接并入 `m.Tr`，就会把大量不透明表面错误识别成玻璃，导致路径大量走折射分支，直接光采样贡献显著减少，最终画面接近全黑。

代码修正为：

- 以 `d/Tr` 决定主要透射强度
- 仅在 `Tf` 明显小于白色且 `Tr` 缺失时才作为 fallback

这个修复直接恢复了 Sponza 的正常 diffuse/NEE 路径。

六、场景支持与测试方式

1. 当前可直接运行的示例场景

当前仓库 `example-scenes-cg25` 下可以直接作为场景目录传给 `PathTracer.exe` 的典型目录包括：

- `cornell-box`
- `living-room`
- `veach-mis`
- `sponza`

其中：

- cornell-box、living-room、veach-mis 是课程场景
- sponza 属于数据集式目录，需要依赖 loader 中的 fallback 逻辑

2. 单场景测试

```
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\cornell-box 16
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\living-room 64
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\sponza 16
```

3. 批量统一测试

仓库中新增了 tools\batch_render_scenes.py，用于自动遍历场景目录并输出统一命名的结果图。

默认行为：

- 扫描 example-scenes-cg25 下所有子文件夹
- 默认渲染 16 64 128 spp
- 输出目录为 rayTracing_report
- 命名规则为 scene_spp.png

例如生成：

- living-room_16.png
- living-room_64.png
- sponza_128.png

使用方式：

```
python tools\batch_render_scenes.py ".\build\windows-msvc-release\PathTracer.exe"
```

```
python tools\batch_render_scenes.py ".\build\windows-msvc-release\PathTracer.exe" --spp 1 2 8 16
```

七、代码阅读路线与模块关系

如果从底层向上阅读，推荐顺序为：

1. src/math/vec.h，src/math/ray.h
2. src/scene/mesh.h
3. src/scene/bvh.h/.cpp
4. src/scene/loader.cpp
5. src/sampling/brdf.cpp
6. src/sampling/light.cpp
7. src/integrator/pathtracer.cpp
8. src/output/tonemap.cpp
9. src/pt_main.cpp

模块关系如下：

```
math(vec/ray)
-> scene(mesh/loader/bvh)
-> sampling(rng/brdf/light)
-> integrator(pathtracer)
-> output(tonemap)
-> pt_main (PathTracer executable)
```

附：常用命令

```
# MSVC 构建
cmake --preset windows-msvc-release
cmake --build --preset windows-msvc-release --parallel

# 单场景渲染
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\living-room 16

# 带像素调试
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\sponza 1 --set PT_DEBUG_PIXEL=640,360

# 输出线性 PFM
.\build\windows-msvc-release\PathTracer.exe .\example-scenes-cg25\living-room 64 --set PT_WRITE_LINEAR=1

# 批量测试
python tools\batch_render_scenes.py ".\build\windows-msvc-release\PathTracer.exe"
```

软光栅化渲染器

一、开发环境与库依赖

本项目基于现代 C++ 标准开发，构建了一个完整的软光栅化渲染器，用于验证和对比不同的消隐算法。

- **操作系统:** Windows 10 / 11 64-bit
- **编程语言:** C++ (ISO C++17 Standard)
- **构建系统:** CMake (跨平台构建脚本)
- **图形接口:** Vulkan SDK (仅作为**显示后端**，用于将 CPU 计算的像素缓冲区提交到显存并显示，核心光栅化逻辑完全由 CPU 实现)
- **第三方库:**
 - **GLFW:** 负责跨平台的窗口创建、上下文管理及键盘/鼠标输入事件处理。
 - **OpenMP:** 用于并行计算加速，特别是在顶点变换 (Vertex Transformation) 和基准 Z-Buffer 的光栅化阶段。
 - **GLM (内嵌实现):** 代码中实现了一个轻量级的数学库 Vec3, Mat4, 用于处理向量运算、矩阵变换及透视投影。

项目部署：

1.编译

#本机存在msvc选择

```
cmake --preset windows-msvc-release
```

```
cmake --build --preset windows-msvc-release
```

#本机存在gcc、g++选择

```
cmake --preset windows-mingw-release
```

```
cmake --build --preset windows-mingw-release
```

#linux暂时没有进行测试和preset的设置，得自行修改一下，抱歉。

2.测试

#漫游式

```
#Usage: ./VulkanApp.exe model.obj [mode] [scenario] [yes/no]
```

```
#mode 1zbuffer 2 scanline zbuffer 3 hzb without bvh 4 full hzb
```

```
#scenario 0是默认场景 其他scenario更多是没有大型obj所强行通过排列补充遮挡情况，容易卡
```

```
#yes no是是否是benchmark环境 no为漫游 默认为no yes为benchmark环境 不可漫游 默认跑1200帧
```

```
#左上显示帧数 红点表示mode 几个红点表示mode几
```

#msvc

```
.\build\windows-msvc-release\VulkanApp.exe .\assets\cubes.obj 1 0 no
```

```
.\build\windows-msvc-release\VulkanApp.exe .\assets\house_t.obj 4 0 yes
```

#gcc

```
.\build\windows-mingw-release\VulkanApp.exe .\assets\cubes.obj 1 0 no
```

```
.\build\windows-mingw-release\VulkanApp.exe .\assets\house_t.obj 4 0 yes
```

```
#获得benchmark_report文件内容如下操作
```

```
#需要python环境存在matplotlib 用来绘制图像
```

```
#控制台会输出每一段的耗时计算 默认测试 scenario0
```

```
python benchmark.py
```

```
#下面容易炸 不同scenario，这里我的排列设计的不是很好 效果一般
```

```
#python benchmark.py --extreme
```

用户操作

程序运行后提供实时的 3D 渲染窗口，支持多种交互模式以便于观察算法效果。

1. 渲染模式切换

benchmark = no(default = no)的状态下，用户可通过键盘数字键实时切换渲染算法，控制台会输出当前的帧率和算法阶段耗时：

- **1 Standard Z-Buffer:** 标准 Z 缓冲算法（基准）。利用 OpenMP 并行加速，暴力遍历三角形包围盒。
- **2 Scanline Z-Buffer:** 扫描线 Z 缓冲算法。利用几何连续性逐行扫描，避免了包围盒法的空像素计算。
- **3 HZB Simple (Linear):** 简单层次 Z 缓冲。无空间结构，线性遍历三角形，逐个查询 HZB 进行剔除。
- **4 HZB Complete (BVH): 完整层次 Z 缓冲。** 结合层次包围盒（BVH），实现了基于节点的整块剔除（Node Culling）。

2. 漫游控制

实现了第一人称自由摄像机：

- **移动:** W / S (前后), A / D (左右), E / Q (升降)。按住 Shift 加速。
- **视角:** 按住鼠标左键拖拽旋转模型/视角，鼠标滚轮缩放视角。
- **动画:** Space 键暂停/恢复模型的自动旋转。

二、 算法实现与模式设计

本项目实现了四种渲染模式，旨在对比传统 Z-Buffer、扫描线算法与层次 Z-Buffer (HZB) 在不同场景下的表现。

1. 渲染模式定义

模式 ID	名称	对应作业要求	算法描述
Mode 1	Standard Z-Buffer	基准对照	传统的 Z-Buffer 算法。利用 OpenMP 并行计算三角形包围盒，暴力光栅化。代表了利用现代多核 CPU 算力的暴力解法。
Mode 2	Scanline Z-Buffer	扫描线算法	改进的光栅化算法。利用几何连续性逐行扫描，避免了空像素计算。在 CPU 单线程逻辑下效率较高，但在并行化方面不如 Mode 1 容易扩展。
Mode 3	HZB Simple	简单模式	无空间结构的 HZB 。线性遍历所有三角形，逐个查询 HZB 进行剔除。流程：Depth Pre-Pass -> Build HZB -> Color Pass (Linear Cull)。
Mode 4	HZB Complete	完整模式	HZB + 层次包围盒 (BVH) 。结合了空间划分。在渲染前，先查询 BVH 节点是否被 HZB 遮挡。流程：Depth Pre-Pass (BVH) -> Build HZB -> Color Pass (Node Cull)。

2. 核心数据结构

1. 层次包围盒 (BVH):
- 采用递归构建，根据三角形重心在最长轴上进行划分。
 - 空间排序 (Spatial Sorting)**: 在渲染遍历时，始终优先访问距离摄像机最近的子节点 (Front-to-Back)，这是 HZB 算法能有效剔除的关键。
2. 层次 Z-Buffer (HZB Pyramid):
- 采用 **Max-Pooling** 构建。第 L 层像素值等于上一层对应 2×2 区域的最大深度值。
 - 允许算法以 $O(1)$ 时间复杂度查询屏幕任意矩形区域的最保守遮挡深度。

三、 实验数据与图表分析

我们在五个不同规模的 OBJ 模型上进行了测试，覆盖了从极低多边形 (Cubes) 到高面数复杂场景 (City)。以下数据基于 Scenario 0 (Base Scenario, 无大量遮挡/实例化)。

1. 总体性能数据 (Scenario 0)

模型	面片数 (Faces)	Mode 1 (Z-Buf)	Mode 2 (Scanline)	Mode 3 (HZB Simple)	Mode 4 (HZB Complete)
Cubes	80	14.12 ms	3.21 ms	16.86 ms	16.73 ms
Teapot	9,216	5.49 ms	2.53 ms	15.63 ms	15.36 ms
Hut	85,644	51.73 ms	13.48 ms	30.73 ms	30.31 ms
House	110,600	30.37 ms	15.05 ms	31.82 ms	30.87 ms

模型	面片数 (Faces)	Mode 1 (Z- Buf)	Mode 2 (Scanline)	Mode 3 (HZB Simple)	Mode 4 (HZB Complete)
City	527,016	24.54 ms	28.24 ms	33.06 ms	33.11 ms

2. 阶段耗时分析 (为何 HZB 变慢?)

观察数据可以发现，在简单场景（Scenario 0）下，**HZB 模式（Mode 3/4）普遍慢于基准算法**。为了深入分析原因，我们拆解了 City.obj (527K 面片) 的耗时结构：

- **Mode 4 (HZB Complete) 耗时拆解:**
 - **Pre-Pass:** 11.04 ms (33%)
 - **HZB Build:** 11.63 ms (35%)
 - **Raster:** 9.17 ms (28%)
 - **Other:** ~1.3 ms (4%)
 - **Total:** 33.11 ms
- **对比 Mode 1 (Z-Buffer):**
 - **Raster:** 23.35 ms (95%)
 - **Total:** 24.54 ms

- 分析:
1. **固定开销 (Overhead):** HZB 算法必须先进行深度预处理 (Pre-Pass) 和金字塔构建 (Build HZB)。在 city 场景中，仅这两项就消耗了 **22.67 ms**，几乎等于 Mode 1 渲染完整一帧的时间。
 2. **光栅化收益 (Raster Gain):** HZB 确实减少了光栅化时间（从 23.35ms 降至 9.17ms，减少了 **60%**）。这证明 HZB 的剔除机制是生效的。
 3. **结论:** 在无严重遮挡的场景下，HZB 节省的光栅化时间不足以抵消其构建成本。这是一个经典的 **“剔除开销 vs. 渲染开销”** 的权衡问题。

四、 算法分析与空间排序的意义

1. 简单模式 (Mode 3) vs. 完整模式 (Mode 4)

虽然在 Scenario 0 的总耗时上两者差异不大，但在算法行为上有本质区别：

- **Mode 3 (Linear):** 必须遍历所有 527,016 个三角形，逐个进行 HZB 查询。Pre-Pass 耗时 7.33ms。
- **Mode 4 (BVH):** 利用 BVH 树。Pre-Pass 耗时 11.04ms（因为递归遍历 BVH 比线性循环更慢，且涉及大量节点包围盒计算）。
- **Cull Time:** Mode 4 记录了 2.20 ms 的 Cull 时间，这意味着它花了时间在树结构上做决策。

为什么 Mode 4 没有显著快于 Mode 3？

在 Scenario 0（普通视角，无大量遮挡）中，大部分物体都是可见的。BVH 无法剔除大块节点（因为节点都在屏幕内且没被遮挡），反而引入了树遍历的开销。Mode 4 的优势通常体现在 "High Depth Complexity" (高深度复杂度) 场景，例如站在一堵墙后面看整个城市，此时 Mode 4 能瞬间剔除 90% 的几何体，而 Mode 3 仍需逐个检查。

2. 物体空间排序 (Spatial Sorting) 的意义

在代码中，我们在遍历 BVH 时实现了 **Front-to-Back (由近及远)** 的排序逻辑：

```
// 优先处理距离相机近的子节点
if (dLeft < dRight) { push(right); push(left); }
else { push(left); push(right); }
```

实验意义:

HZB 是一种 保守剔除 算法。它依赖于 Z-Buffer 中已经存在的深度值来遮挡后续物体。

- **如果乱序渲染:** 算法可能先处理了远处的建筑 A。此时 Z-Buffer 是空的，A 被画了出来。接着处理近处的建筑 B。虽然 B 挡住了 A，但 A 已经画了，计算量已经浪费了。
- **空间排序后:** 算法先处理近处的建筑 B，填入较浅的深度值。当处理远处的建筑 A 时，查询 HZB 发现 A 的深度大于 B，直接剔除 A。

数据佐证:

在 City 模型中，尽管总时间变长，但 Mode 4 的 Raster 时间 (9.17ms) 远低于 Mode 1 (23.35ms) 和 Mode 2 (26.91ms)。这证明通过空间排序，我们成功利用近处的物体遮挡了远处的物体，避免了大量的像素着色计算 (Overdraw)。

3. 扫描线算法 (Mode 2) 的表现

- **优势:** 在 Cubes, Teapot, Hut, House 等中小规模模型上，**Scanline 算法是最快的** (比 Z-Buffer 快 2-5 倍)。这是因为扫描线算法只处理三角形覆盖的有效像素，完全避免了包围盒算法中对空像素的无效遍历与测试。
- **劣势:** 在 city (527K 面片) 这种超大规模场景下，Scanline 的串行逻辑 (难以像 Mode 1 那样简单地 OpenMP 并行化) 成为了瓶颈，被暴力的并行 Z-Buffer 反超。

五、 结论

1. 算法效率:

- **低负载/中负载:** 扫描线 Z-Buffer (Mode 2) 效率最高，它是 CPU 软光栅化的理想选择。
- **高负载 (海量面片):** 简单的并行 Z-Buffer (Mode 1) 凭借多核优势胜出。

2. HZB 的适用性:

- HZB 算法引入了显著的固定开销 (Pre-Pass + Build)。在简单场景 (Scenario 0) 下，这种开销会导致性能下降 (加速比 < 1.0)。
- 然而，HZB 成功减少了 **60% 以上的光栅化时间**。这表明在 **像素着色开销极大** (如复杂 Shader) 或 **遮挡极高** (如室内漫游、城市建筑群) 的场景下，HZB 将从“负优化”转变为“性能救星”。

3. 空间排序:

- 是 HZB 算法生效的基石。没有由近及远的排序，HZB 只能作为 Z-Buffer 的一种昂贵替代品，而无法发挥其“提早剔除”的核心优势。

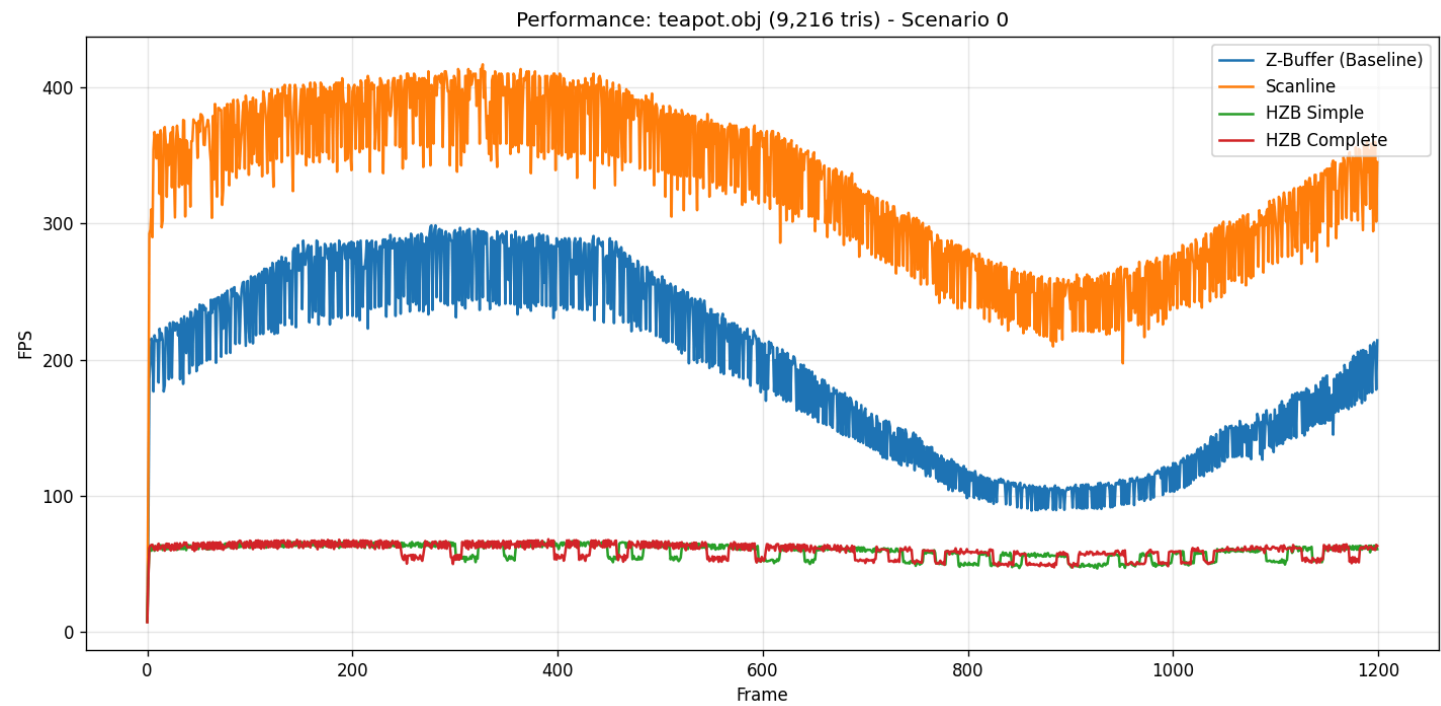
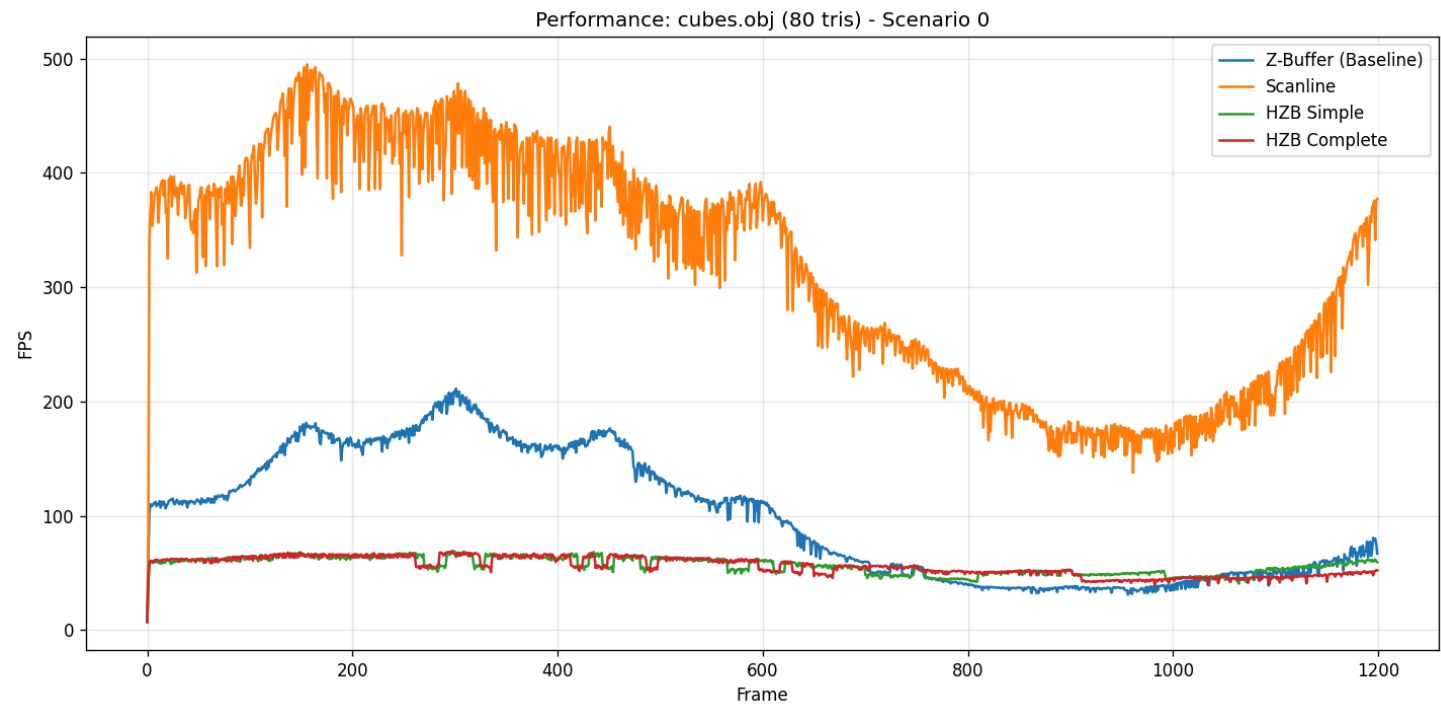
其中我其实考虑过一些并行优化光栅化操作，但是比较容易出现数据竞争，并且最终效果并不稳定，后续再CG2 光线追踪的时候再回过头来考虑优化。

这里的最终结论有点可惜，因为我没有找到更好的比如1000k面片情况下是否会出现更显著的反超情况，后续会补充实验内容，和完善代码。

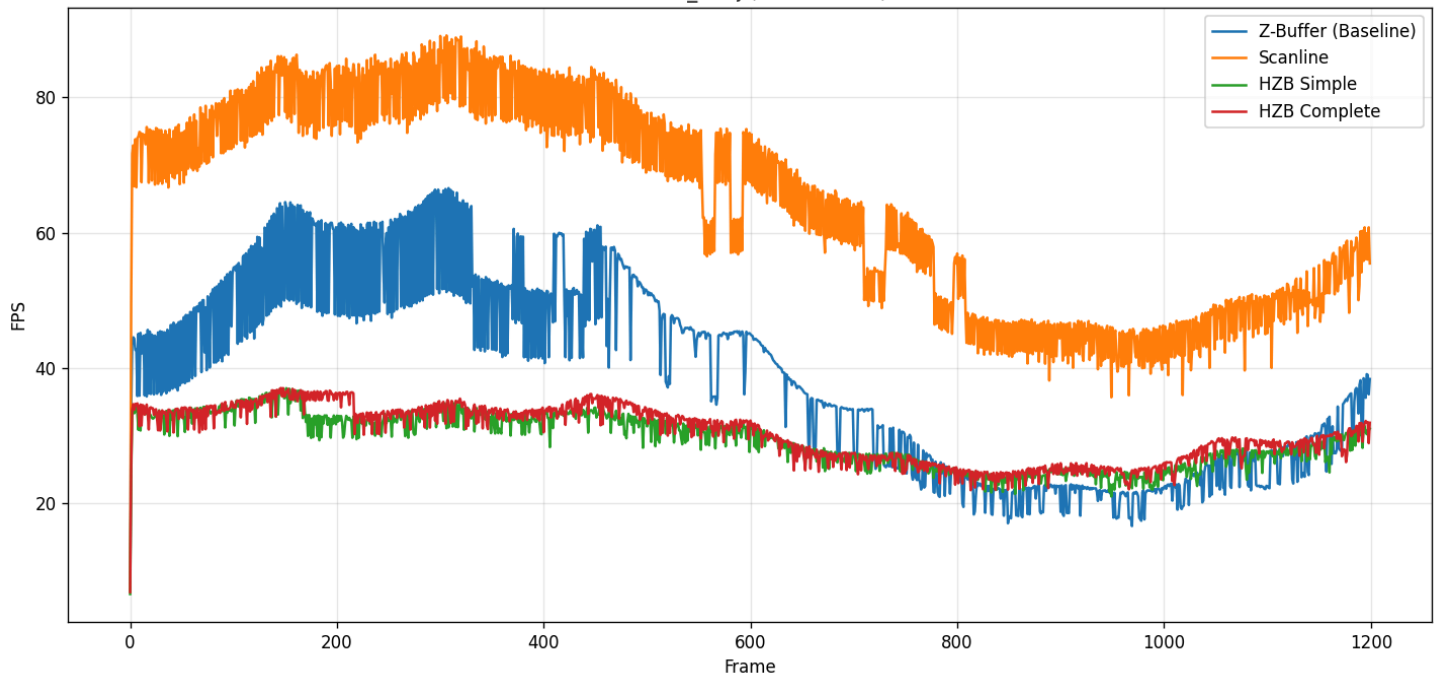
benchmark测试：

实验环境:Win11\Intel(R)Core(TM)Ultra 7 265KF

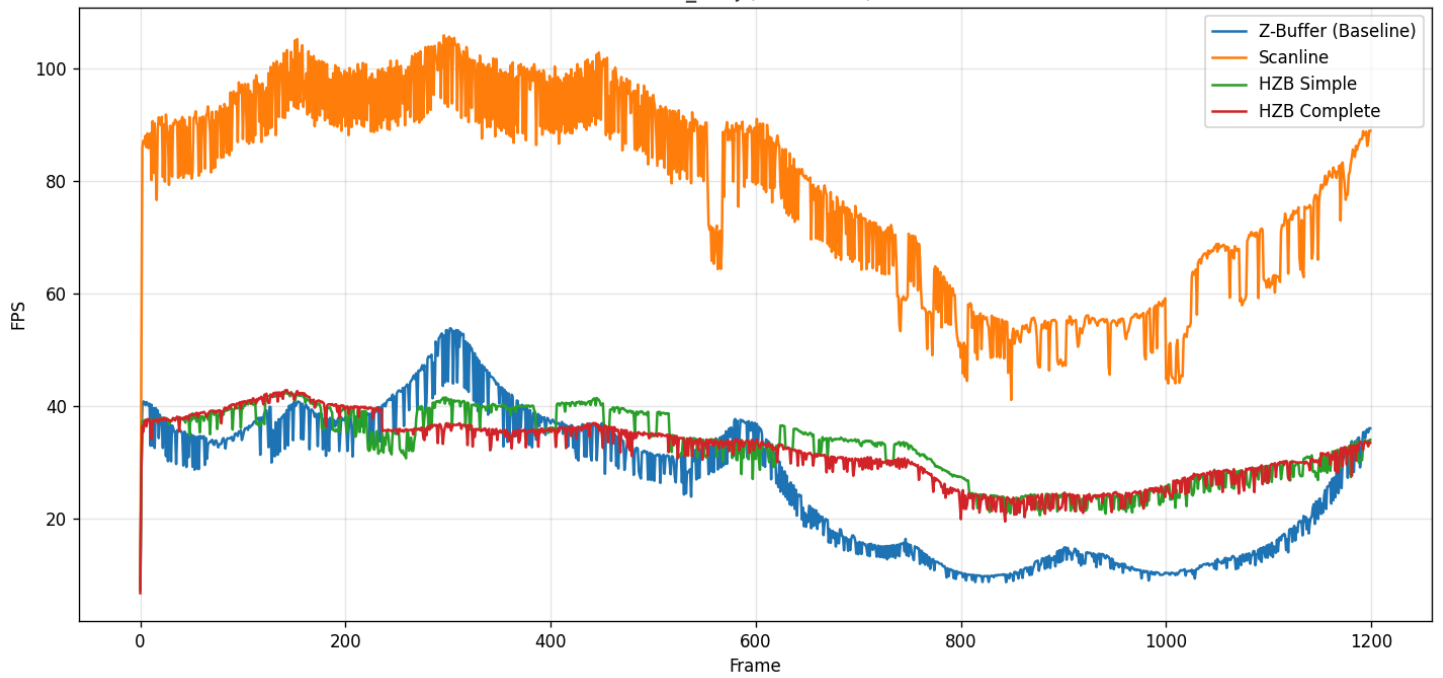
以下为scenario 0 即默认排列的情况

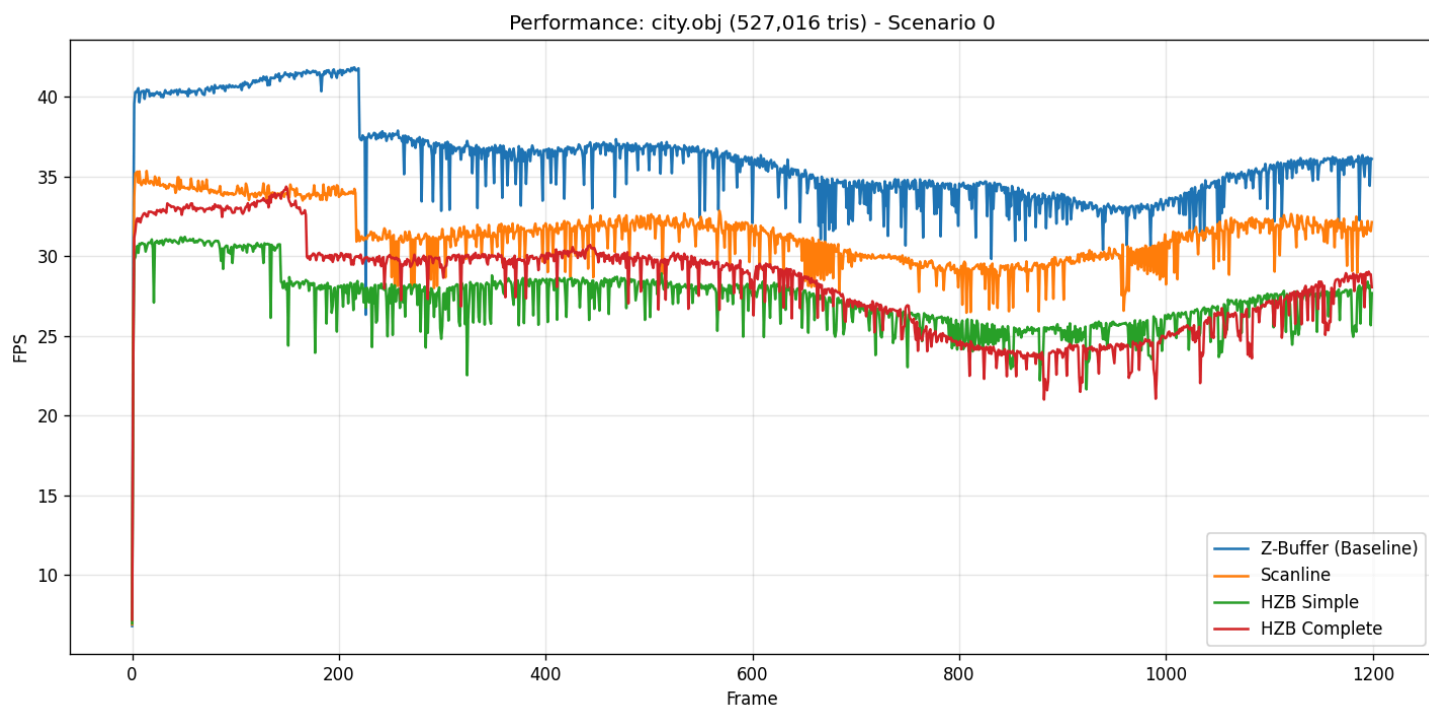


Performance: house_t.obj (110,600 tris) - Scenario 0



Performance: hut_t.obj (85,644 tris) - Scenario 0





scenario 3 mode 1 house_t.obj 仅作展示情况



(cv) C:\Users\lcknight\Desktop\CG\CGAssignment>python benchmark.py
Target Executable: C:\Users\lcknight\Desktop\CG\CGAssignment\build\windows-msvc-release\VulkanApp.exe
Scanning models in 'assets'...
Found 5 models:
- cubes.obj | Faces: 80
- teapot.obj | Faces: 9,216
- hut_t.obj | Faces: 85,644
- house_t.obj | Faces: 110,600
- city.obj | Faces: 527,016

[===== Testing Model: cubes.obj (80 faces) =====]

--- Stage Breakdown (Scenario 0) ---

Mode	Clear	Vertex	PrePass	HZB	Cull	Raster	Total (ms)	FPS
Z-Buffer (Baseline)	0.86	0.01	0.00	0.00	0.00	13.24	14.12	100.7
Scanline	0.75	0.00	0.00	0.00	0.00	2.46	3.21	316.5
HZB Simple	0.79	0.01	1.45	12.48	0.00	2.12	16.86	56.7
HZB Complete	0.80	0.01	1.46	12.45	0.00	2.01	16.73	56.0

Saved FPS chart: benchmark_report\cubes.obj_fps.png

[===== Testing Model: teapot.obj (9,216 faces) =====]

--- Stage Breakdown (Scenario 0) ---

Mode	Clear	Vertex	PrePass	HZB	Cull	Raster	Total (ms)	FPS
Z-Buffer (Baseline)	0.77	0.00	0.00	0.00	0.00	4.72	5.49	195.0
Scanline	0.77	0.00	0.00	0.00	0.00	1.76	2.53	330.1
HZB Simple	0.92	0.00	1.31	12.08	0.00	1.31	15.63	59.3
HZB Complete	0.94	0.00	1.41	11.76	0.09	1.25	15.36	59.9

Saved FPS chart: benchmark_report\teapot.obj_fps.png

[===== Testing Model: hut_t.obj (85,644 faces) =====]

--- Stage Breakdown (Scenario 0) ---

Mode	Clear	Vertex	PrePass	HZB	Cull	Raster	Total (ms)	FPS
Z-Buffer (Baseline)	0.94	0.03	0.00	0.00	0.00	50.76	51.73	26.4
Scanline	0.94	0.02	0.00	0.00	0.00	12.52	13.48	78.7
HZB Simple	0.98	0.02	8.42	11.84	0.00	9.48	30.73	32.9
HZB Complete	0.91	0.02	8.39	11.81	0.02	9.18	30.31	32.1

Saved FPS chart: benchmark_report\hut_t.obj_fps.png

[===== Testing Model: house_t.obj (110,600 faces) =====]

--- Stage Breakdown (Scenario 0) ---

Mode	Clear	Vertex	PrePass	HZB	Cull	Raster	Total (ms)	FPS
Z-Buffer (Baseline)	0.98	0.03	0.00	0.00	0.00	29.36	30.37	38.9
Scanline	0.85	0.02	0.00	0.00	0.00	14.17	15.05	64.4
HZB Simple	0.94	0.02	9.78	11.70	0.00	9.37	31.82	29.4

HZB Complete
 | 0.92
 | 0.02
 | 9.67
 | 11.69
 | 0.21
 | 8.56
 | 30.87
 | 30.2

Saved FPS chart: benchmark_report\house_t.obj_fps.png

[===== Testing Model: city.obj (527,016 faces) =====]

--- Stage Breakdown (Scenario 0) ---

Mode	Clear	Vertex	PrePass	HZB	Cull	Raster	Total (ms)	FPS
Z-Buffer (Baseline)	1.04	0.15	0.00	0.00	0.00	23.35	24.54	36.1
Scanline	1.17	0.15	0.00	0.00	0.00	26.91	28.24	31.4
HZB Simple	1.11	0.18	7.33	11.63	0.00	12.82	33.06	27.3
HZB Complete	1.11	0.16	11.04	11.63	2.20	9.17	33.11	28.2

Saved FPS chart: benchmark_report\city.obj_fps.png

Benchmark Suite Completed.